

Recovery

Dr. Geetha Mary A
Professor,
SCOPE, VIT
Vellore, India

Database Recovery

1 Purpose of Database Recovery

- To bring the database into the last consistent state, which existed prior to the failure.
- To preserve transaction properties (Atomicity, Consistency, Isolation and Durability).
- Example:
 - If the system crashes before a fund transfer transaction completes its execution, then either one or both accounts may have incorrect value. Thus, the database must be restored to the state before the transaction modified any of the accounts.

Database Recovery

2 Types of Failure

- The database may become unavailable for use due to
 - **Transaction failure:** Transactions may fail because of incorrect input, deadlock, incorrect synchronization.
 - **System failure:** System may fail because of addressing error, application error, operating system fault, RAM failure, etc.
 - **Media failure:** Disk head crash, power disruption, etc.

Database Recovery

3 Transaction Log

- For recovery from any type of failure data values prior to modification (BFIM - BeFore Image) and the new value after modification (AFIM – AFTer Image) are required.
- These values and other information is stored in a sequential file called Transaction log. A sample log is given below. Back P and Next P point to the previous and next log records of the same transaction.

T ID	Back P	Next P	Operation	Data item	BFIM	AFIM
T1	0	1	Begin			
T1	1	4	Write	X	X = 100	X = 200
T2	0	8	Begin			
T1	2	5	W	Y	Y = 50	Y = 100
T1	4	7	R	M	M = 200	M = 200
T3	0	9	R	N	N = 400	N = 400
T1	5	nil	End			

Database Recovery

4 Data Update

- **Immediate Update:** As soon as a data item is modified in cache, the disk copy is updated.
- **Deferred Update:** All modified data items in the cache is written either after a transaction ends its execution or after a fixed number of transactions have completed their execution.
- **Shadow update:** The modified version of a data item does not overwrite its disk copy but is written at a separate disk location.
- **In-place update:** The disk version of the data item is overwritten by the cache version.

Database Recovery

5 Data Caching

- Data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk.
- The flushing is controlled by **Modified** and **Pin-Unpin** bits.
 - **Pin-Unpin**: Instructs the operating system not to flush the data item.
 - **Modified**: Indicates the AFIM of the data item.

Database Recovery

6 Transaction **Roll-back (Undo)** and **Roll-Forward (Redo)**

- To maintain atomicity, a transaction's operations are redone or undone.
 - **Undo:** Restore all BFIMs on to disk (Remove all AFIMs).
 - **Redo:** Restore all AFIMs on to disk.
- Database recovery is achieved either by performing only Undos or only Redos or by a combination of the two. These operations are recorded in the log as they happen.

Database Recovery

Write-Ahead Logging

- When **in-place** update (immediate or deferred) is used then log is necessary for recovery and it must be available to recovery manager. This is achieved by **Write-Ahead Logging (WAL)** protocol. WAL states that
 - **For Undo:** Before a data item's AFIM is flushed to the database disk (overwriting the BFIM) its BFIM must be written to the log and the log must be saved on a stable store (log disk).
 - **For Redo:** Before a transaction executes its commit operation, all its AFIMs must be written to the log and the log must be saved on a stable store.

Database Recovery

7 Checkpointing

- Time to time (randomly or under some criteria) the database flushes its buffer to database disk to minimize the task of recovery. The following steps defines a checkpoint operation:
 1. Suspend execution of transactions temporarily.
 2. Force write modified buffer data to disk.
 3. Write a [checkpoint] record to the log, save the log to disk.
 4. Resume normal transaction execution.
- During recovery redo or undo is required to transactions appearing after [checkpoint] record.

Database Recovery

Steal/No-Steal and Force/No-Force

- Possible ways for flushing database cache to database disk:
 1. Steal: Cache can be flushed before transaction commits.
 2. No-Steal: Cache cannot be flushed before transaction commit.
 3. Force: Cache is immediately flushed (forced) to disk.
 4. No-Force: Cache is deferred until transaction commits
- These give rise to four different ways for handling recovery:
 - Steal/No-Force (Undo/Redo)
 - Steal/Force (Undo/No-redo)
 - No-Steal/No-Force (Redo/No-undo)
 - No-Steal/Force (No-undo/No-redo)

Database Recovery

- Deferred Update (No Undo/Redo)
- Deferred Update in a single-user system
There is no concurrent data sharing in a single user system. The data update goes as follows:
 - A set of transactions records their updates in the log.
 - At commit point under WAL scheme these updates are saved on database disk.
- After reboot from a failure the log is used to redo all the transactions affected by this failure. No undo is required because no AFIM is flushed to the disk before a transaction commits.

Database Recovery

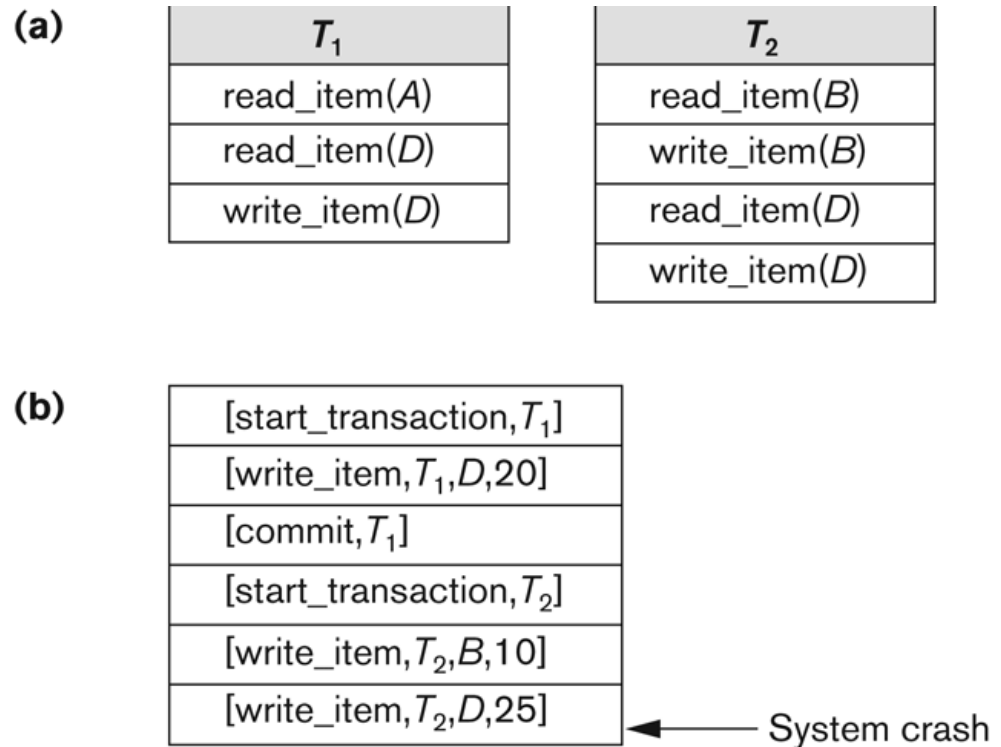


Figure 19.2

An example of recovery using deferred update in a single-user environment. (a) The READ and WRITE operations of two transactions. (b) The system log at the point of crash.

The [write_item,...] operations of T_1 are redone.

T_2 log entries are ignored by the recovery process.

Database Recovery

Deferred Update with concurrent users

- This environment requires some concurrency control mechanism to guarantee **isolation** property of transactions. In a system recovery transactions which were recorded in the log after the last checkpoint were **redone**. The recovery manager may scan some of the transactions recorded before the checkpoint to get the AFIMs.

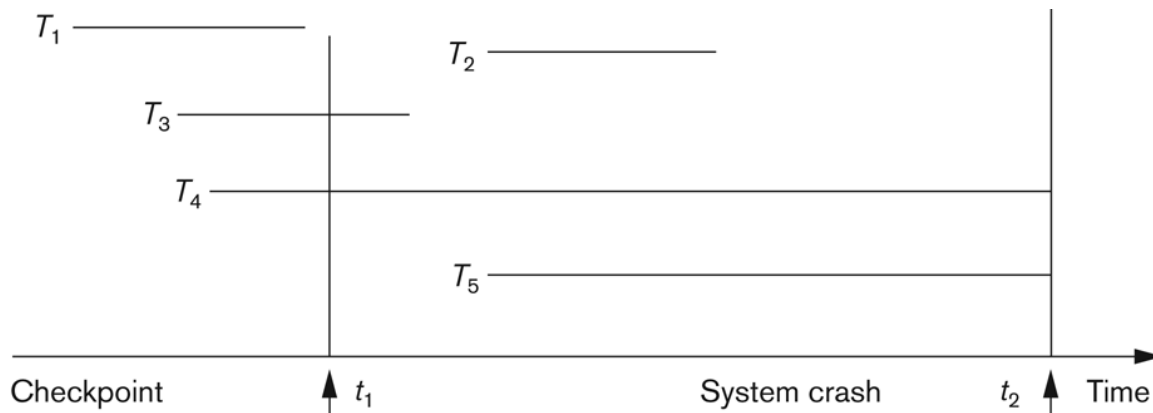
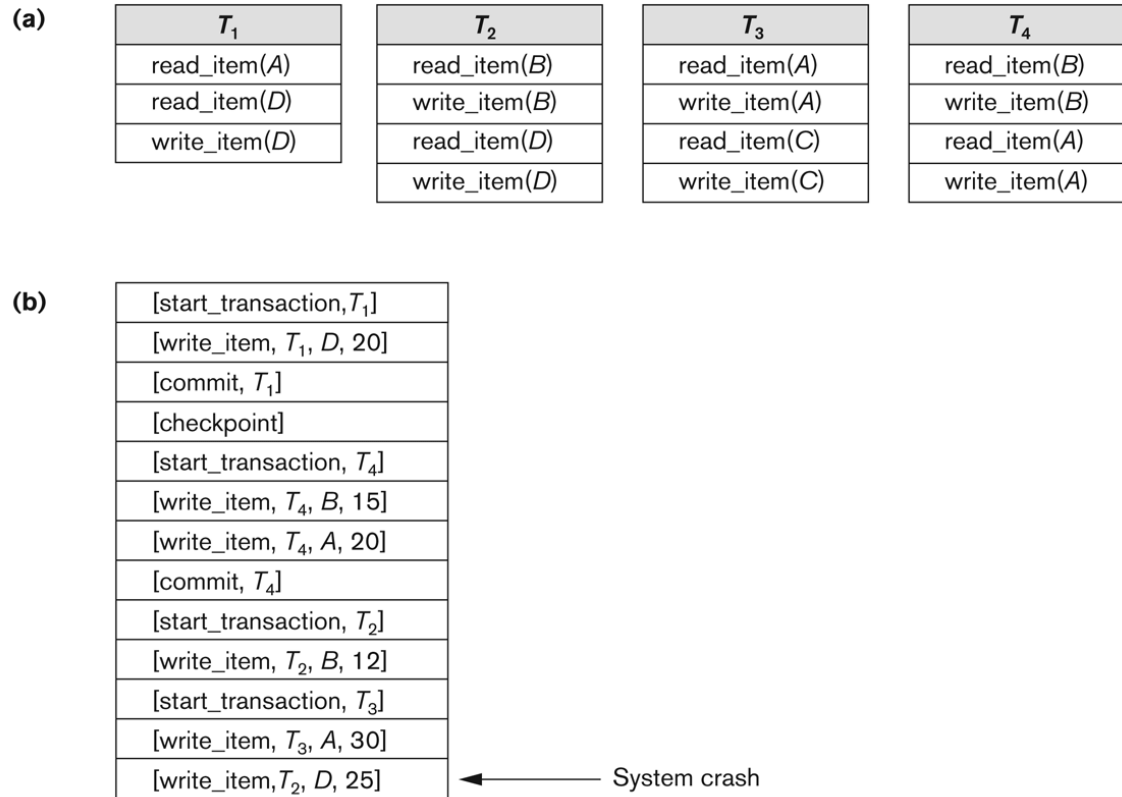


Figure 19.3
An example of recovery in a multi-user environment

Database Recovery



T_2 and T_3 are ignored because they did not reach their commit points.

T_4 is redone because its commit point is after the last system checkpoint.

Figure 19.4

An example of recovery using deferred update with concurrent transactions.

(a) The READ and WRITE operations of four transactions. (b) System log at the point of crash.

Database Recovery

Deferred Update with concurrent users

- Two tables are required for implementing this protocol:
 - **Active table:** All active transactions are entered in this table.
 - **Commit table:** Transactions to be committed are entered in this table.
- During recovery, all transactions of the **commit** table are redone and all transactions of **active** tables are ignored since none of their AFIMs reached the database. It is possible that a **commit** table transaction may be **redone** twice but this does not create any inconsistency because of a redone is “**idempotent**”, that is, one redone for an AFIM is equivalent to multiple redone for the same AFIM.

Database Recovery

Recovery Techniques Based on Immediate Update

■ Undo/No-redo Algorithm

- In this algorithm AFIMs of a transaction are flushed to the database disk under WAL before it commits.
- For this reason the recovery manager **undoes** all transactions during recovery.
- No transaction is **redone**.
- It is possible that a transaction might have completed execution and ready to commit but this transaction is also **undone**.

Database Recovery

Recovery Techniques Based on Immediate Update

■ **Undo/Redo Algorithm (Single-user environment)**

- Recovery schemes of this category apply **undo** and also **redo** for recovery.
- In a single-user environment no concurrency control is required but a log is maintained under WAL.
- Note that at any time there will be one transaction in the system and it will be either in the commit table or in the active table.
- The recovery manager performs:
 - **Undo** of a transaction if it is in the **active** table.
 - **Redo** of a transaction if it is in the **commit** table.

Database Recovery

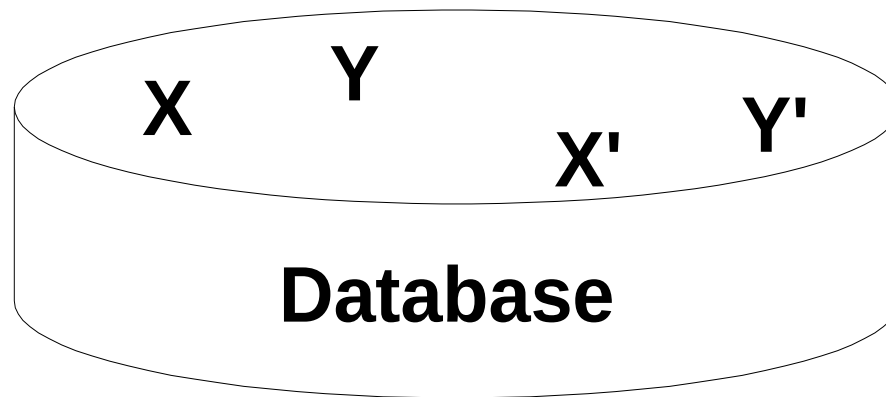
Recovery Techniques Based on Immediate Update

- **Undo/Redo Algorithm (Concurrent execution)**
- Recovery schemes of this category applies **undo** and also **redo** to recover the database from failure.
- In concurrent execution environment a concurrency control is required and log is maintained under WAL.
- Commit table records transactions to be committed and active table records active transactions. To minimize the work of the recovery manager checkpointing is used.
- The recovery performs:
 - **Undo** of a transaction if it is in the **active** table.
 - **Redo** of a transaction if it is in the **commit** table.

Database Recovery

Shadow Paging

- The AFIM does not overwrite its BFIM but recorded at another place on the disk. Thus, at any time a data item has AFIM and BFIM (Shadow copy of the data item) at two different places on the disk.



X and Y: Shadow copies of data items

X' and Y': Current copies of data items

Database Recovery

Shadow Paging

- To manage access of data items by concurrent transactions two directories (current and shadow) are used.
 - The directory arrangement is illustrated below. Here a page is a data item.

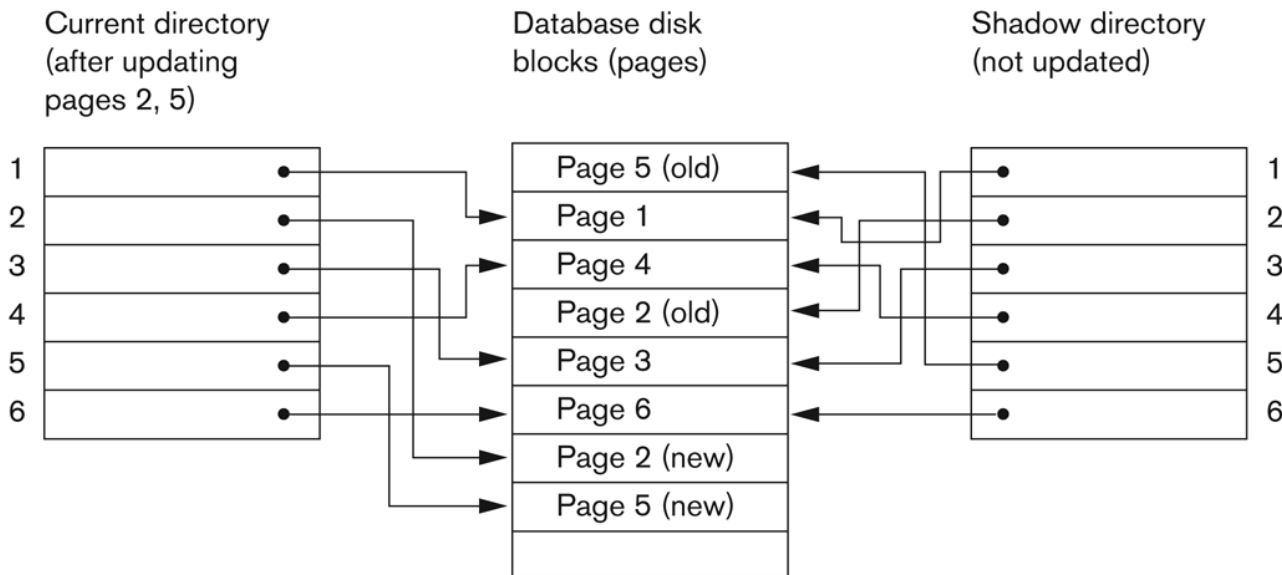


Figure 19.5
An example of shadow paging.